

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB RECORD

Bio Inspired Systems (23CS5BSBIS)

Submitted by

MAKADIA RISHIT DILIPBHAI (1BM23CS177)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Aug-2025 to Dec-2025

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “ Bio Inspired Systems (23CS5BSBIS)” carried out by **MAKADIA RISHIT DILIPBHAI (1BM23CS177)**, who is a bonafide student of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements of the above mentioned subject and the work prescribed for the said degree.

Radhika A.D. Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE
---	--

Index

Sl. No.	Date	Experiment Title	Page No.
1	18/8/25	Genetic Algorithm (Lab_1)	2
2	1/9/25	Particle Swarm Optimisation (Lab_2)	7
3	8/9/25	Ant Colony Optimisation (Lab_3)	10
4	15/9/25	Cuckoo Search Optimisation (Lab_4)	14
5	29/9/25	Grey Wolf Optimisation (Lab_5)	18
6	13/10/25	Parallel Cellular Algorithm (Lab_6)	23
7	25/8/25	Gene Expression Algorithm (Lab_7)	29

Github Link:

https://github.com/rishitmakadia/1BM23CS177_BIS_5

Program 1

Implement a Genetic Algorithm in Python to find the maximum value of a simple function $f(x) = x^2$.

Algorithm:

Lab-1: Genetic Algorithm for Optimization Problems

Algorithm:

Step-1: Define the problem
Define the function which needs to be optimized which is known as "fitness function"
Eg:- $f(x) = \cos(x)$, $0 < x < 1$, $f(x) = x^2$

Step-2: Initialize Parameters
There are few parameters that needs to be defined.
Population Count $\rightarrow$ No. of solⁿ that exist at one generation
Mutation Rate $\rightarrow$ prob. of value changing randomly.
Crossover Rate $\rightarrow$ prob. that two parents will cross
No. of Generation $\rightarrow$ No. of iterations to be done.

Step-3: Initial Population
Initialize the first iteration of population randomly.
i.e. value of x randomly [0,1]

String No.	Offspring after Crossover	Mutation	Offspring after Mutation	x Value	Fitness $f(x) = x^2$
1	01101	10000	11101	29	841
2	11000	00000	11000	24	576
3	11011	00000	11011	27	729
4	10001	00101	10100	20	400
$\Sigma f(x)$					2546
Avg					636.5
Max					841

Step-4: Evaluate Fitness
Putting the value of x in fitness fuⁿ $f(x) = x^2$

Step-5: Selection
Here, the values are kept in a roulette wheel selection with value with higher fitness score, having higher chance of selection.

Step-6: Crossover
Two parents will cross their genes, & the crossover point is chosen random.

Step-7: Mutation
Based on changed crossover, the mutation value is increased or decreased & the fitness function is calculated again.

Output:
Best Solution Before Any Generation
Genome: 11110, Integer: 30, Fitness: 900

Generation 1:
Avg. Fitness: 644, Max. Fitness: 900
Before Mutation
Genome: 11110, Integer: 30, Fitness: 900
Genome: 11000, Integer: 24, Fitness: 576
After Mutation
Genome: 11100, Integer: 28, Fitness: 784
Genome: 11100, Integer: 28, Fitness: 784

Generation 2:
Avg. Fitness: 682, Max. Fitness: 784
After Mutation
Genome: 11011, Integer: 27, Fitness: 729
Genome: 10100, Integer: 20, Fitness: 400

Code:

```
import random
import numpy as np
# pip install numpy

# 1. Define the Problem
def fitness_function(x):
    return x**2

# 2. Initialize Parameters
```

```

POPULATION_SIZE = 4
GENOME_LENGTH = 5 #bits
# 1 = Every pair of parents will definitely crossover.
CROSSOVER_RATE = 0.5
# 1 = Every bit of every child genome will flip.
MUTATION_RATE = 1
GENERATIONS = 3

# Decode to integer
def decode(genome):
    return int(genome, 2)

# random 10110 type once
def generate_genome():
    return "".join(random.choice('01') for _ in
range(GENOME_LENGTH))

# random 10110 type for entire population_size
def create_population():
    return [generate_genome() for _ in
range(POPULATION_SIZE)]

def evaluate(population):
    # Evaluate fitness on decoded integer (x) evaluates
    fitness function
    return [fitness_function(decode(ind)) for ind in
population]

# selects two individuals (parents) based on their fitness
probabilities
def select(population, fitnesses):
    total_fit = sum(fitnesses)
    probs = [f / total_fit for f in fitnesses]
    return random.choices(population, weights=probs, k=2)

# Combines two parent genomes (p1 and p2) to create two
offspring by exchanging genetic material at a random
crossover point.
def crossover(p1, p2):
    if random.random() < CROSSOVER_RATE:
        point = random.randint(1, GENOME_LENGTH - 1)

```

```

        return p1[:point] + p2[point:], p2[:point] + p1[point:]
    else:
        return p1, p2

def mutate(genome):
    # Flip exactly 1 bit at a random position
    pos = random.randint(0, GENOME_LENGTH - 1)
    bit_list = list(genome)
    bit_list[pos] = '1' if bit_list[pos] == '0' else '0'
    return ''.join(bit_list)

def genetic_algorithm():
    population = create_population()

    # Show the best individual before any generation or
    # mutation
    fitnesses = evaluate(population)
    best_initial = max(zip(population, fitnesses),
        key=lambda x: x[1])
    best_initial_genome, best_initial_fit = best_initial
    best_initial_x = decode(best_initial_genome)
    print("\n=== Best Solution Before Any Generation ===")
    print(f"Genome: {best_initial_genome}")
    print(f"Decoded Integer: {best_initial_x}")
    print(f"Fitness (x^2): {best_initial_fit:.4f}\n")

    for generation in range(1, GENERATIONS + 1):
        fitnesses = evaluate(population)

        # Sort population by fitness descending
        pop_fit = list(zip(population, fitnesses))
        pop_fit.sort(key=lambda x: x[1], reverse=True)

        # Calculate avg and max fitness
        avg_fitness = sum(fitnesses) / len(fitnesses)
        max_fitness = max(fitnesses)

        print(f"\n--- Generation {generation} ---")
        print(f"Average Fitness: {avg_fitness:.4f}")
        print(f"Max Fitness: {max_fitness:.4f}")

```

```

# Show top 2 individuals BEFORE mutation
print("Top 2 individuals BEFORE mutation:")
for i, (genome, fit) in enumerate(pop_fit[:2], 1):
    decoded = decode(genome)
    print(f' Rank {i}: Genome={genome} | Decoded
Integer={decoded} | Fitness={fit:.4f} ")

new_population = []

while len(new_population) < POPULATION_SIZE:
    parent1, parent2 = select(population, fitnesses)
    child1, child2 = crossover(parent1, parent2)

    # Mutate children with exactly 1 bit flip
    mutated_child1 = mutate(child1)
    mutated_child2 = mutate(child2)

    new_population.extend([mutated_child1,
mutated_child2])

population = new_population[:POPULATION_SIZE]

# Evaluate new population after mutation
new_fitnesses = evaluate(population)

# Sort new population by fitness descending
new_pop_fit = list(zip(population, new_fitnesses))
new_pop_fit.sort(key=lambda x: x[1], reverse=True)

# Show top 2 individuals AFTER mutation
print("Top 2 individuals AFTER mutation:")
for i, (genome, fit) in enumerate(new_pop_fit[:2], 1):
    decoded = decode(genome)
    print(f' Rank {i}: Genome={genome} | Decoded
Integer={decoded} | Fitness={fit:.4f} ")

# After all generations, output overall best
overall_best = max(new_pop_fit, key=lambda x: x[1])
best_genome, best_fit = overall_best
best_x = decode(best_genome)

```

```
    print("\n=== Best Solution Found After All Generations  
===")  
    print(f"Genome: {best_genome}")  
    print(f"Decoded Integer: {best_x}")  
    print(f"Fitness (x^2): {best_fit}")  
  
genetic_algorithm()
```


Program 2

Implement the Particle Swarm Optimization (PSO) algorithm to minimize the De Jong function:

$$f(x, y) = x^2 + y^2$$

Algorithm:

The image shows handwritten notes on lined paper detailing the steps of the Particle Swarm Optimization (PSO) algorithm. The notes are organized into five steps, with the first four steps on the left page and the fifth step and output on the right page.

Step 1: Define the Problem
Specify the objective function to minimize (e.g., De Jong function $f(x) = x^2 + y^2$)

Step 2: Set Parameters
Choose the parameters:
• number of particles in the swarm
• maximum no. of iterations
• inertia weight
• cognitive coefficient (c_1)
• social coefficient (c_2)

Step 3: Initialize the Swarm
For each particle in the swarm:
• Randomly initialize the particle's position within the bounds, velocity
• Evaluate & store particles personal best fitness score using the objective function (De Jong)

Step 4: Determine the Global Best
Find the particle with the lowest personal best fitness & set its position as the global best position, with its score as the global best score

Step 5: Iterative Optimization Loop
For each particle:
• update velocity
calculate the new velocity using w, c_1, c_2
i.e. inertia, cognitive influence & social influence
• update position
Add the new velocity to the current position & ensure it is within bounds.
• Evaluate Fitness
calculate fitness score at the new position.
• Update Personal Best
if the new fitness score is better than the particle's personal best score, update the personal best position & score

Update Global Best
After updating all particles, if any particles best score is better than global then update it.

Output:

Iteration	Global Best
1/10	6.8900
2/10	1.4074
3/10	0.0132
4/10	0.0132
5/10	0.0132
6/10	0.0132
7/10	0.0132
8/10	0.004320
9/10	0.004320
10/10	0.002113

Best Position: $T = 0.01077, -0.002097$

Code:

```
import random
```

```
# Step 1: Define the De Jong fitness function
```

```
def de_jong(position):
```

```
    x, y = position
```

```
    return x**2 + y**2
```

```
# Step 2: Particle class
```

```
class Particle:
```

```
    def __init__(self, bounds):
```

```
        self.position = [random.uniform(low, high) for low, high in bounds]
```

```

self.velocity = [random.uniform(-1, 1) for _ in bounds]
self.best_position = list(self.position)
self.best_score = de_jong(self.position)

def update_velocity(self, global_best_position, w, c1, c2):
    for i in range(len(self.velocity)):
        r1 = random.random()
        r2 = random.random()
        cognitive = c1 * r1 * (self.best_position[i] - self.position[i])
        social = c2 * r2 * (global_best_position[i] - self.position[i])
        self.velocity[i] = w * self.velocity[i] + cognitive + social

def update_position(self, bounds):
    for i in range(len(self.position)):
        self.position[i] += self.velocity[i]
        # Clamp to bounds
        self.position[i] = max(bounds[i][0], min(self.position[i], bounds[i]
[1]))

    current_score = de_jong(self.position)
    if current_score < self.best_score:
        self.best_score = current_score
        self.best_position = list(self.position)

# Step 3: PSO Algorithm
def pso(
    function,
    bounds=[(-10, 10), (-10, 10)],
    num_particles=30,
    max_iter=10,
    w=0.5,    # inertia weight
    c1=1.5,   # personal influence | Cognitive Coefficient
    c2=1.5    # best influence | Social Coefficient
):
    # Initialize particles
    swarm = [Particle(bounds) for _ in range(num_particles)]

    # Initialize global best
    global_best_position = min(swarm, key=lambda p:
p.best_score).best_position
    global_best_score = function(global_best_position)

```

```

# Main loop
for iteration in range(max_iter):
    for particle in swarm:
        particle.update_velocity(global_best_position, w, c1, c2)
        particle.update_position(bounds)

    # Update global best
    for particle in swarm:
        if particle.best_score < global_best_score:
            global_best_score = particle.best_score
            global_best_position = list(particle.best_position)

    print(f'Iteration {iteration+1}/{max_iter} - Global Best:
{global_best_score:.6f}')

    print("\n✅ Optimization Complete!")
    print(f'Best Position: {global_best_position}')
    print(f'Best Score (f(x,y)): {global_best_score:.6f}')
    return global_best_position, global_best_score

# Step 4: Run the PSO
if __name__ == "__main__":
    pso(de_jong)

```

Program 3

Implement the Ant Colony Optimization (ACO) algorithm to find the shortest possible route that visits a set of cities exactly once and returns to the starting city — i.e., solve the Travelling Salesman Problem (TSP).

Algorithm:

8/9/25

classmate
Date _____
Page _____

Lab-3 Ant Colony Optimization for the Travelling Salesman Problem

Algorithm:

Step 1: Define the Problem
We first define a set of cities with their respective coordinates. For each pair of cities, the distance matrix is calculated using Euclidean distance.

Step 2: Initialize parameters:

- Number of ants (num ants)
- Number of Iteration (iterations) : no. of times ants construct their path
- Pheromone Importance (alpha) : influence of pheromone levels on the ant's decision.
- Huristic Importance (beta) : influence of (inverse of distance) on the ants decisions.
- Pheromone Evaporation Rate (rho) : controls how quickly the pheromone evaporation, reducing their influence on the
- Pheromone Deposit factor (q) : The amount of pheromone deposited on path

Step 3: Construct Solutions:

- Each ant starts at a randomly selected city.
- Ant construct its tour by choosing the next city based on a probability influenced by : α alpha & β beta.

$$P_{ij} = \frac{[\tau_{ij}]^{\alpha} \cdot [\eta_{ij}]^{\beta}}{\sum_{k \in \text{neighbours}} [\tau_{ik}]^{\alpha} \cdot [\eta_{ik}]^{\beta}}$$

τ_{ij} : Pheromone level on edge (i,j)
 $\eta_{ij} = 1/d_{ij}$: heuristic info.

Step 4: Update Pheromones
Once all ants have completed the tour, pheromones are updated. Influenced by : ρ rho $\rightarrow$ Evaporation.
 q $\rightarrow$ Pheromone Deposit.
Shortest path receives more pheromones.

Step 5: Iterate.
The process is repeated for fixed no. of iterations.
After each iteration, the best solution found so far is updated.

Step 6: Output the Best Solution

Output

Iteration 1/10	-	Best distance	= 32.70
" 2/10	-	"	= 33.51
" 3/10	-	"	= 33.10
" 4/10	-	"	= 32.77
" 5/10	-	"	= 32.55
" 6/10	-	"	= 32.37
" 7/10	-	"	= 32.20
" 8/10	-	"	= 32.20
" 9/10	-	"	= 32.20
" 10/10	-	"	= 32.20

best dist of (A) & (B) w $\frac{1}{2} P = 2$

classmate
Date _____
Page _____

8/9/25

Lab-3

Step 4: Update Pheromones

Step 5: Iterate

Step 6: Output the Best Solution

Output

Iteration 1/10 - Best distance = 32.70

" 2/10 - " " = 33.51

" 3/10 - " " = 33.10

" 4/10 - " " = 32.77

" 5/10 - " " = 32.55

" 6/10 - " " = 32.37

" 7/10 - " " = 32.20

" 8/10 - " " = 32.20

" 9/10 - " " = 32.20

" 10/10 - " " = 32.20

TSP / TSP studying

Code:

```
import numpy as np
import random

# Define number of cities and their coordinates
cities = [ [0, 0], [1, 5], [5, 2], [3, 8], [7, 7],
           [9, 0], [6, 5], [4, 6], [8, 3], [2, 1]]

# Number of ants, iterations, pheromone parameters
```

```

num_ants = 5000
iterations = 10
alpha = 1 # Pheromone importance (increased a little)
beta = 1 # Heuristic importance (1/distance)
rho = 0.1 # Pheromone evaporation rate (moderate value)
q = 10 # Pheromone deposit factor (increased for stronger influence)
# Number of cities
num_cities = len(cities)

# Create distance matrix
distances = np.zeros((num_cities, num_cities))
for i in range(num_cities):
    for j in range(num_cities):
        if i != j:
            distances[i][j] = np.linalg.norm(np.array(cities[i]) - np.array(cities[j]))

# Initialize pheromone matrix with a small constant value (not too high)
pheromone = np.ones((num_cities, num_cities)) * 0.1 # Lower initial pheromone values

# Precompute heuristic (inverse of distance)
heuristic = 1 / (distances + np.eye(num_cities)) # Avoid division by zero on diagonal
np.fill_diagonal(heuristic, 0)

# Function to construct a solution (tour) for each ant
def construct_solution():
    path = []
    visited = set()

    # Randomly choose a starting city
    current_city = random.randint(0, num_cities - 1)
    path.append(current_city)
    visited.add(current_city)

    while len(path) < num_cities:
        probabilities = calculate_transition_probabilities(current_city, visited)
        next_city = np.random.choice(range(num_cities), p=probabilities)
        path.append(next_city)
        visited.add(next_city)
        current_city = next_city

    return path

```

```

# Function to calculate transition probabilities for the ants
def calculate_transition_probabilities(current_city, visited):
    probabilities = np.zeros(num_cities)
    pheromone_strength = pheromone[current_city]
    heuristic_info = heuristic[current_city]

    # Calculate probability for each city based on pheromone and heuristic
    for city in range(num_cities):
        if city not in visited:
            probabilities[city] = (pheromone_strength[city] ** alpha) * (heuristic_info[city] ** beta)

    total_prob = np.sum(probabilities)
    if total_prob == 0:
        unvisited = [city for city in range(num_cities) if city not in visited]
        for city in unvisited:
            probabilities[city] = 1.0
        total_prob = len(unvisited)

    return probabilities / total_prob

# Function to calculate the total distance of a given path
def calculate_distance(path):
    distance = 0
    for i in range(len(path)):
        from_city = path[i]
        to_city = path[(i + 1) % num_cities] # wrap around to start
        distance += distances[from_city][to_city]
    return distance

# Function to update pheromone levels based on solutions
def update_pheromones(all_paths, all_distances):
    global pheromone
    pheromone *= (1 - rho) # Evaporate pheromone

    # Update pheromones for each path based on its quality (distance)
    for path, distance in zip(all_paths, all_distances):
        pheromone_deposit = q / distance
        for i in range(len(path)):
            from_city = path[i]
            to_city = path[(i + 1) % num_cities]

```

```

    pheromone[from_city][to_city] += pheromone_deposit
    pheromone[to_city][from_city] += pheromone_deposit # symmetric update

# Normalize pheromone levels and add slight randomness to prevent stagnation
pheromone = pheromone / np.sum(pheromone) # Normalize to avoid infinite growth
pheromone += np.random.rand(num_cities, num_cities) * 0.01 # Small noise for exploration

# Main loop to run the Ant Colony Optimization
best_distance = float('inf')
best_path = None

for iteration in range(iterations):
    all_paths = []
    all_distances = []

    # Each ant constructs a solution
    for ant in range(num_ants):
        path = construct_solution()
        distance = calculate_distance(path)
        all_paths.append(path)
        all_distances.append(distance)

    if distance < best_distance:
        best_distance = distance
        best_path = path

# Update pheromones based on solutions found by ants
update_pheromones(all_paths, all_distances)

# Output the best distance at this iteration
print(f'Iteration {iteration + 1}/{iterations} - Best distance: {best_distance:.2f}')

# Output the best path and its distance
print("\nBest path found:", best_path)
print("Best path distance:", round(best_distance, 2))

```

Program 4

Implement the Cuckoo Search Optimization Algorithm (CSO) to find the minimum of the Sphere function, defined as:

$$f(x) = \sum x^2$$

Algorithm:

classmate
Date _____
Page _____

15/11/25

Lab 4: Cuckoo Search (Vehicle Routing Problem)

Algorithm:

Step 1: Define the problem.
Minimize the total dist. for a fleet of vehicles delivering goods to customers, starting and ending at a central depot.

Step 2: Initialize Parameters

- No. of Nests (N): no. of solution (routes)
- Max. No. of Iteration (max. it): max. attempt for optimization
- Prob. of discovery (p_n): fraction of worst routes to abandon & replace
- No. of Vehicles: total vehicles available for the routing
- No. of Customer: total customers to be served
- Vehicle capacity: max. customer on vehicle
- Depot coordinate: Coordinates of the central depot

Step 3: Initialize Population (Nests)

- Generate an initial population of random routes
- Each route will be a list of customers assigned to diff. vehicles

Step 4: Evaluate Fitness

- The fitness function evaluates the total distance travelled by all vehicles in the given set of routes.
- Objective fn: Compute the total distance of the routes by summing the distance b/w consecutive customers of the depot
- Priority: If vehicle exceed its capacity
- Fitness value: The solⁿ with the smallest distance (total route dist)

Distance $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$ Fitness = $\sum$ Distance + Penalty
Penalty: Excess Flight = Random value (val)

Vehicle Routing Problem
Fitness Function

classmate
Date _____
Page _____

Step 5: Generate New Solution

- Randomly choose a route & perform the 2-opt swap
- Pick two random cust. posⁿ in a route
- Swap the cust. b/w the selected position

Step 6: Abandon worst Nests

- After each iteration, sort the nests by fitness value (routes from best to worst)
- Abandon a fraction of the worst-performing nests

Step 7: Iterate & Optimize

- Repeat the following process for fixed iteration
- Evaluate fitness
- Track Best solution
- Replace worst Nests
- Print progress

Step 8: Output the Best Solution

Our best solution will be the set of routes for all vehicles with the minimum total dist. travelled.

classmate
Date _____
Page _____

Output:

Iteration 0: Best Fitness = 664.077

Iteration	Best Fitness
1	218.96
2	587.31
3	586.50
4	506.26
5	506.26
6	506.26
7	506.26
8	506.26
9	506.26

Best Solution Found:

Vehicle 1: Distance = 101.25

Vehicle	Distance
2	101.25
3	101.25
4	101.25
5	101.25

Don't forget to check the results

Code:

```
import numpy as np
import random

# Step 1: Define the objective function (for
optimization)
def objective_function(x):
    # Example: Sphere function (minimization)
    return np.sum(x**2)

# Step 2: Initialize parameters
n_nests = 50 # Number of nests
max_iter = 100 # Maximum number of
iterations
pa = 0.25 # Probability of discovery (egg
being discovered by host)
n_dim = 10 # Number of dimensions (problem
size)
alpha = 1.5 # Scaling factor for Lévy flights

# Step 3: Initialize population (random
positions)
nests = np.random.uniform(-10, 10, (n_nests,
n_dim))

# Step 4: Evaluate fitness (objective function
value for each nest)
fitness =
np.apply_along_axis(objective_function, 1,
nests)

# Step 5: Lévy flight approximation (using a
simplified version without external
dependencies)
def levy_flight(n_dim, alpha=1.5):
    # Simplified Lévy flight step using a random
walk with a power law
    # We use a normal distribution for this
approximation, though it's less accurate than
the original Levy distribution
```

```

u = np.random.normal(0, 1, n_dim)
v = np.random.normal(0, 1, n_dim)
step = u / np.power(np.abs(v), 1 / alpha)
return step

# Step 6: Abandon worst nests and replace
them with new ones
def replace_worst_nests(nests, fitness, pa,
max_iter):
    # Sort nests based on fitness values
    sorted_indices = np.argsort(fitness)
    best_nests = nests[sorted_indices[:n_nests //
2]]
    worst_nests = nests[sorted_indices[n_nests //
2:]]

    # Abandon the worst nests and replace them
for i in range(n_nests // 2):
    if random.random() < pa:
        # Generate a new nest using Lévy flight
        step = levy_flight(n_dim, alpha)
        new_nest = best_nests[i] + step
        new_nest = np.clip(new_nest, -10, 10)
# Ensure within bounds
        nests[sorted_indices[n_nests // 2 + i]] =
new_nest
    return nests

# Step 7: Iterate and perform optimization
best_solution = None
best_fitness = float('inf')

for iteration in range(max_iter):
    # Evaluate fitness of each nest
    fitness =
np.apply_along_axis(objective_function, 1,
nests)

    # Find the best solution
    min_fitness_idx = np.argmin(fitness)
    min_fitness_value = fitness[min_fitness_idx]

```

```

if min_fitness_value < best_fitness:
    best_fitness = min_fitness_value
    best_solution = nests[min_fitness_idx]

# Replace worst nests with new solutions
nests = replace_worst_nests(nests, fitness,
pa, max_iter)

# Print progress (every 10 iterations)
# if iteration % 10 == 0:
print(f'Iteration {iteration}: Best Fitness =
{best_fitness}')

# Step 8: Output the best solution
print("\nBest Solution Found:")
print(f'Position: {best_solution}')
print(f'Fitness: {best_fitness}')

```

Program 5

Implement the Grey Wolf Optimizer (GWO) algorithm to perform clustering on a set of 2D data points.

The algorithm should optimize the positions of cluster centroids such that the sum of squared distances between data points and their nearest centroids is minimized (similar to K-Means).

Algorithm:

24/9/25

Lab 5 Grey Wolf Optimizer (GWO)

Algorithm: (K-means clustering) used in GWO

Step 1: Define the Problem
The goal is to find the optimal centroids for clustering a set of data points using the K-means algo.
Fitness fun = sum of squared dist. from each data point to its closest centroid

Step 2: Initialize Parameters
num wolves (W) = no. of wolves
num. of iterations =
K = (Number of clusters)
dim = K * 2 (Dimension of each centroid)
→ no. centroids
Search Space → Lower-bound
→ upper-bound

Step 3: Initialize Population
- Initialize random position of all the wolves in the given search space.
- Position of α, β, δ along with scores
Alpha (α) : best solution found so far
Beta (β) : Second best solution
Delta (δ) : Third best solution

Step 4: Fitness Function
Fitness = $\sum_{i=1}^n \min_j \|x_i - c_j\|^2$ → Population Distance
↓
data point centroid

Step 5: Update Position (Encircling & Hunting)
 $dist_{i,j} = \frac{x_1 + x_2 + x_3}{3}$
Random coefficient $A = 2 \cdot a \cdot r_1 - a$ $a = 2 \cdot r_2$
 $dist_{i,j} = (c_1 \cdot \alpha - dist_{i,j})$

Step 6: Main Loop
- Update the α, β, δ values & their fitness.
- Track the best fitness value (α) & corresponding centroid.

Step 7: Output
Print the best centroids, plot the convergence curve.

Output

Iteration 1

α Fitness	: 313.77
β Fitness	: 673.16
δ Fitness	: 787.14
α "	: 837.93

Iteration 5

α "	: 189.89
β "	: 209.34
δ "	: 223.61
α "	: 242.76

Best Fitness value : 189.89
best centroid [2.836, 3.000].

Code:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Generate random data points
np.random.seed(42)
num_points = 100
X = np.vstack([
```

```

    np.random.normal(loc=[3, 3], size=(num_points//2, 2)), #
Cluster 1
    np.random.normal(loc=[7, 7], size=(num_points//2, 2)) #
Cluster 2
])

# Number of clusters (K)
K = 2

# GWO Parameters
num_wolves = 30
num_iterations = 5
dim = K * 2 # Each centroid has 2 coordinates (x, y)

# Search space boundaries
lower_bound = 0
upper_bound = 10

# Initialize wolves' positions (randomly within the search space)
wolves = np.random.uniform(lower_bound, upper_bound,
                             (num_wolves, dim))

# Initialize alpha, beta, delta, omega wolves
alpha_pos = np.zeros(dim)
alpha_score = float('inf')

beta_pos = np.zeros(dim)
beta_score = float('inf')

delta_pos = np.zeros(dim)
delta_score = float('inf')

omega_pos = np.zeros(dim)
omega_score = float('inf')

# To track fitness over iterations (for convergence curve)
fitness_curve = []

# Euclidean distance function
def euclidean_distance(x1, x2):
    return np.linalg.norm(x1 - x2)

```

```

# Fitness function (K-means objective: sum of squared distances
to centroids)
def fitness(wolves, X, K):
    wolf_centroids = wolves.reshape((K, 2))
    total_distance = 0
    for point in X:
        # Calculate the distance to the closest centroid
        distances = [euclidean_distance(point, centroid) for centroid
in wolf_centroids]
        total_distance += min(distances)**2 # Sum of squared
distances
    return total_distance

# Clip wolf positions to remain within boundaries
def clip_positions(wolves, lb, ub):
    np.clip(wolves, lb, ub, out=wolves)

# Main GWO loop
for iter in range(num_iterations):
    for i in range(num_wolves):
        # Calculate the fitness of each wolf (position)
        fit = fitness(wolves[i], X, K)

        # Update alpha, beta, delta, omega positions based on fitness
        if fit < alpha_score:
            alpha_score = fit
            alpha_pos = wolves[i].copy()
        elif fit < beta_score:
            beta_score = fit
            beta_pos = wolves[i].copy()
        elif fit < delta_score:
            delta_score = fit
            delta_pos = wolves[i].copy()
        elif fit < omega_score:
            omega_score = fit
            omega_pos = wolves[i].copy()

    # Calculate "a" for the hunting equation (linearly decreasing
from 2 to 0)
    a = 2 - iter * (2 / num_iterations)

```

```

# Update the positions of all wolves based on the encircling and
# hunting equations
for i in range(num_wolves):
    for j in range(dim):
        r1 = np.random.rand()
        r2 = np.random.rand()

        # Encircling & Hunting Equations
        A1 = 2 * a * r1 - a
        C1 = 2 * r2
        D_alpha = np.abs(C1 * alpha_pos[j] - wolves[i, j])
        X1 = alpha_pos[j] - A1 * D_alpha

        r1 = np.random.rand()
        r2 = np.random.rand()

        A2 = 2 * a * r1 - a
        C2 = 2 * r2
        D_beta = np.abs(C2 * beta_pos[j] - wolves[i, j])
        X2 = beta_pos[j] - A2 * D_beta

        r1 = np.random.rand()
        r2 = np.random.rand()

        A3 = 2 * a * r1 - a
        C3 = 2 * r2
        D_delta = np.abs(C3 * delta_pos[j] - wolves[i, j])
        X3 = delta_pos[j] - A3 * D_delta

        # Update wolf's position based on the three best wolves
        wolves[i, j] = (X1 + X2 + X3) / 3

# Clip wolf positions to ensure they remain within the defined
# boundaries
clip_positions(wolves, lower_bound, upper_bound)

# Record the best fitness value (for convergence curve)
fitness_curve.append(alpha_score)

```

```

    # Print positions of Alpha, Beta, Delta, Omega wolves at each
iteration
    print(f"Iteration {iter + 1}:")
    print(f" Alpha Wolf Position: {alpha_pos} | Fitness:
{alpha_score}")
    print(f" Beta Wolf Position: {beta_pos} | Fitness:
{beta_score}")
    print(f" Delta Wolf Position: {delta_pos} | Fitness:
{delta_score}")
    print(f" Omega Wolf Position: {omega_pos} | Fitness:
{omega_score}")
    print("-" * 50)

# Final solution (best centroids) is the position of the alpha wolf
print(f"Best solution found with fitness value: {alpha_score}")
alpha_pos = alpha_pos.reshape((K, 2))
print("Best centroids (Alpha Wolf):")
for idx, centroid in enumerate(alpha_pos):
    print(f"Centroid {idx + 1}: {centroid}")

# Plotting the final result of clustering with the best centroids
plt.figure(figsize=(8, 8))

# Plot data points
plt.scatter(X[:, 0], X[:, 1], c='blue', label='Data Points', alpha=0.6)

# Plot the centroids
for idx, centroid in enumerate(alpha_pos):
    plt.scatter(centroid[0], centroid[1], color='red', s=100,
label=f'Centroid {idx + 1}')

plt.xlabel('X')
plt.ylabel('Y')
plt.title('Clustering Result with Grey Wolf Optimizer')
plt.legend()
plt.grid(True)
plt.show()

```


Program 6

Implement a Parallel Cellular Algorithm for Network Design and Analysis.

The objective is to find an optimal subset of network edges that connects all nodes with minimum total cost, while maintaining connectivity.

Disconnected networks are penalized in the fitness function.

Algorithm:

13/10/25

Lab-6 Parallel Cellular Algorithms & Programmes

Algorithm

Step 1: Define the Problem
Formulate the Network Design & Analysis optimization task.
Each potential solution (cell) represents a subset of edges connecting the given set of nodes.

Step 2: Initialize Parameters
Grid size (size of parallel cellular grid)
Grid size (GxG): no. of cells (candidate solutions)
Neighborhood structure (NBS): define how cell interacted.
No. of iterations (T):
Penalty factors: Used to penalize disconnected n/w
Selection rules: define how new candidate topologies are generated.

Step 3: Initialize population:
For each cell c_{ij} in the GxG grid:
• Randomly generate an initial network topology $E_{ij} \subseteq E$
• Each cell's state thus represents a different possible n/w config.

Step 4: Evaluate Fitness:
Fitness value: $Fitness(E_{ij}) = \sum cost(e)$
If n/w not connected, apply a pf
 $Fitness(E_{ij}) = \text{Total Cost} + \text{Penalty Factor} \times (\text{Components} - 1)$
Lower Fitness value $\rightarrow$ better solution.

(u,v, cost)

Adv: Parallelism, Distributed optimization, Scalability

Step 5: Update States:
• Collect n/w topologies from its neighboring cells defined by
• Combine these with its own topology to form candidate pool of n/w
• Generate new candidate topologies, favoring lower fitness value
• Evaluate & update the fitness value $E_{ij} \leftarrow E'_{ij}$

Step 6: Iterate
Repeat the evaluation & state update steps for T iterations.
[Throughout this process, information spreads across the grid, allowing promising designs to influence their neighbors & evolve collaboratively.]

Step 7: Output the Best Solution:
• Identify the cell with the lowest fitness value, representing best n/w config.
• Output the corresponding subset of edges of the total optimized n/w

Output: node=6, G=3

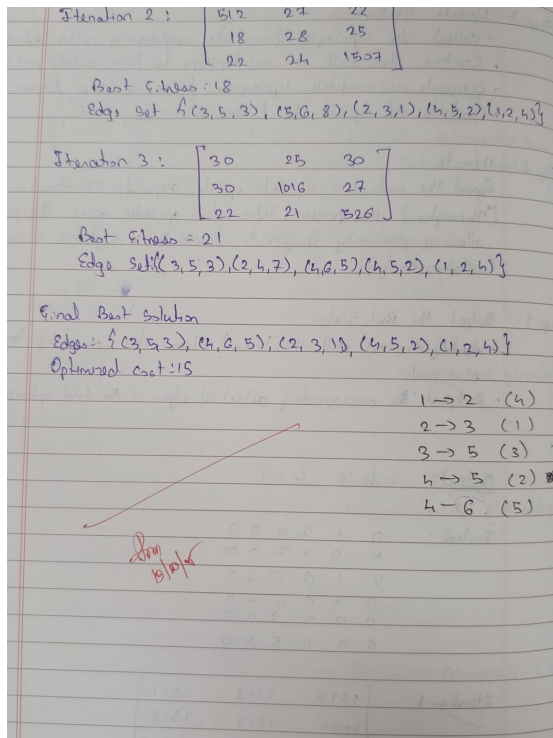
Initial:

0	4	2	0	0	0
4	0	1	7	0	0
2	1	0	0	3	0
0	7	0	0	2	5
0	0	3	2	0	0
0	0	0	5	3	0

Iteration 1:

1512	2518	2501
2004	1513	2508
2508	2508	916

Best fitness value = 916
Edge set: $\{(4,5,2), (3,5,3), (2,4,7), (1,2,4)\}$



Code:

```
import random
import math
from collections import defaultdict, deque
```

```
# -----
# Helper Functions
# -----
```

```
def is_connected(nodes, edges):
```

```
    """Check if the given edges connect all nodes (using BFS)."""
```

```
    if not edges:
```

```
        return False, len(nodes)
```

```
    adj = defaultdict(list)
```

```
    for u, v, _ in edges:
```

```
        adj[u].append(v)
```

```
        adj[v].append(u)
```

```
    visited = set()
```

```
    q = deque([nodes[0]])
```

```
    while q:
```

```
        n = q.popleft()
```

```
        if n not in visited:
```

```

        visited.add(n)
        for nei in adj[n]:
            if nei not in visited:
                q.append(nei)
    components = len(nodes) - len(visited) + 1
    return len(visited) == len(nodes), components

```

```

def fitness(nodes, edges, penalty_factor=1000):
    """Compute total cost with penalty for disconnected networks."""
    total_cost = sum(c for _, _, c in edges)
    connected, components = is_connected(nodes, edges)
    if not connected:
        total_cost += penalty_factor * (components - 1)
    return total_cost

```

```

def von_neumann_neighbors(i, j, G, wrap=True):
    """Return von Neumann neighbors of cell (i,j)."""
    neighbors = []
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    for di, dj in directions:
        ni, nj = i + di, j + dj
        if wrap:
            ni %= G
            nj %= G
        if 0 <= ni < G and 0 <= nj < G:
            neighbors.append((ni, nj))
    return neighbors

```

```

# -----
# Main Algorithm
# -----

```

```

def parallel_cellular_network_design(nodes, edges, G=4, T=10, penalty_factor=1000):
    """
    Parallel Cellular Algorithm for Network Design and Analysis.
    Displays grid state, fitness values, and best solution at each iteration.
    """
    # Initialize random population

```

```

grid = [[set(random.sample(edges, random.randint(1, len(edges)//2)))
         for _ in range(G)] for _ in range(G)]

for t in range(T):
    fitness_grid = [[0]*G for _ in range(G)]

    # Evaluate fitness
    for i in range(G):
        for j in range(G):
            f = fitness(nodes, grid[i][j], penalty_factor)
            fitness_grid[i][j] = f

    # Find best fitness & corresponding edge in this iteration
    best_iter_fit = math.inf
    best_iter_edges = None
    for i in range(G):
        for j in range(G):
            if fitness_grid[i][j] < best_iter_fit:
                best_iter_fit = fitness_grid[i][j]
                best_iter_edges = grid[i][j]

    # Display iteration info
    print(f"\n===== Iteration {t+1} =====")
    print("Fitness Matrix:")
    for row in fitness_grid:
        print([" {:.1f} ".format(x) for x in row])

    print(f"\nBest Fitness This Iteration: {best_iter_fit:.2f}")
    print(f"Best Edge Set: {best_iter_edges}")

    # Update states (in parallel logic)
    new_grid = [[set() for _ in range(G)] for _ in range(G)]
    for i in range(G):
        for j in range(G):
            # Gather neighbors
            neighbors = von_neumann_neighbors(i, j, G)
            candidate_pool = set(grid[i][j])
            for ni, nj in neighbors:
                candidate_pool.update(grid[ni][nj])

            candidate_list = list(candidate_pool)

```

```

    if not candidate_list:
        new_grid[i][j] = set()
        continue

    # Generate multiple candidates from the pool
    candidates = []
    for _ in range(5): # sample 5 new candidates
        k = random.randint(1, len(candidate_list))
        subset = random.sample(candidate_list, k)
        candidates.append(set(subset))

    # Choose best candidate
    best_edges = min(candidates, key=lambda e: fitness(nodes, e, penalty_factor))
    new_grid[i][j] = best_edges

grid = new_grid

# Final best solution after all iterations
best_edges = None
best_fit = math.inf
for i in range(G):
    for j in range(G):
        f = fitness(nodes, grid[i][j], penalty_factor)
        if f < best_fit:
            best_fit = f
            best_edges = grid[i][j]

print("\n===== Final Best Solution =====")
print("Edges:", best_edges)
print("Optimized Cost:", best_fit)

# -----
# Example Usage
# -----

if __name__ == "__main__":
    # Example network with 6 nodes and edges with costs
    nodes = [1, 2, 3, 4, 5, 6]
    edges = [
        (1, 2, 4), (1, 3, 2), (2, 3, 1),

```

```
(2, 4, 7), (3, 5, 3), (4, 5, 2),  
(4, 6, 5), (5, 6, 8)  
]
```

```
parallel_cellular_network_design(nodes, edges, G=3, T=3, penalty_factor=500)
```

Program 7

Optimize the search efficiency of multiple drones exploring a grid by coordinating their positions based on neighbors' success using a Parallel Cellular Algorithm to maximize the total searched area.

Algorithm:

Optimization via Gene Expression Algorithms

Algorithm:

Step 1: The problem is to solve the Traveling Salesperson Problem (TSP). The goal is to find the minimized distance to visit all nodes & back to source.

Step 2: Initialize Parameters
Population Size (POP_SIZE): No. of solution in each generation.
No. of Genes: total number of cities (NUM_CITIES)
Mutation Rate (MUTATION_RATE): prob. of random change in route.
Crossover Rate (NUM_PARENTS): no. of parents (cities) selected to create the next generation.
Number of Generation (MAX_GENERATIONS): max. no. of iterations the algorithm will run.

Step 3: Initialize Population (random genetic sequence)
Each sequence is a random permutation of the city indices, representing a unique tour.

Step 4: Evaluate fitness
For each genetic sequence (route), evaluate its fitness.
$$\text{Fitness} = \frac{1}{\text{route distance}}$$

i.e. A shorter tour results in a higher fitness score.

Step 5: Selection
Select genetic sequences (parents) for reproduction based on their fitness.
This is done using Roulette wheel Selection, where routes with higher fitness have higher chance of being chosen to create the next generation.

Step 6: Crossover. (New pair of selected parents to produce new offspring)
A random subsection from the first parent is copied to the child, & the remaining cities are filled in from the second parent in order they appear, ensuring the child is a valid tour.

Step 7: Mutation (this introduces new variations into the population)
For each route, there is a small probability that two randomly chosen cities (genes) in the sequence will be swapped.

Step 8: Gene Expression
The genetic sequence is directly interpreted as the function solution (which is the exact order the salesperson visits the cities).

Step 9: Repeatable
Repeat the selection, crossover, mutation, & evaluation processes for a fixed number of generations.
The process can stop early if the best solution does not improve for a set no. of generations.

Step 10: Output the Best Solution
Throughout the iterations, continuously track the route with the highest fitness. Output the overall best route & its total distance as the optimal solution found by the algorithm.

Output:-
Initial best distance: 1837.07
Generation 1: Best distance = 1676.01
Generation 9: " = 1550.23
Stopping early before 50 gen.
Genetic Algorithm Finished
Optimal Route: [5 19 4 12 7 - - - - - 3 1 2 6]
Total distance = 1550.23

Code:

```
import random
import math

# --- 1. Define cities with (x, y) coordinates ---
# For demonstration, we'll create some random cities.
# In a real application, you would load these from a file.
```

```

NUM_CITIES = 20
CITIES = [(random.randint(0, 200),
random.randint(0, 200)) for _ in
range(NUM_CITIES)]

# --- Helper Function: distance ---
def distance(city1, city2):
    """Return Euclidean distance between city1
    and city2."""
    return math.sqrt((city1[0] - city2[0])**2 +
(city1[1] - city2[1])**2)

# --- Helper Function: tourDistance ---
def tour_distance(route):
    """
    Calculate the total distance of a tour.
    A route is a list of city indices, e.g., [0, 3, 1,
    2, ...].
    """
    total_dist = 0
    num_cities = len(route)
    for i in range(num_cities):
        # Get the distance between the current
        city and the next city
        from_city = CITIES[route[i]]
        # If it's the last city, the next city is the
        starting city
        to_city = CITIES[route[(i + 1) %
num_cities]]
        total_dist += distance(from_city, to_city)
    return total_dist

# --- Helper Function: fitness ---
def fitness(route):
    """
    Calculate the fitness of a route. Fitness is
    the inverse of the tour distance.
    A shorter distance means a higher fitness.
    """
    return 1 / tour_distance(route)

```



```

# --- GA Function: initializePopulation ---
def initialize_population(pop_size,
num_cities):
    """Create 'popSize' random routes, each
visiting all cities exactly once."""
    population = []
    base_route = list(range(num_cities))
    for _ in range(pop_size):
        route = random.sample(base_route,
len(base_route)) # Create a random
permutation
        population.append(route)
    return population

# --- GA Function: calculateFitness (for the
whole population) ---
def calculate_all_fitness(population):
    """For each route in the population,
compute its fitness."""
    return [fitness(route) for route in
population]

# --- GA Function: selectMatingPool ---
def select_mating_pool(population,
fitness_scores, num_parents):
    """Use roulette wheel selection to pick
'numParents' routes."""
    # random.choices allows for weighted
selection with replacement
    parents = random.choices(
        population=population,
        weights=fitness_scores,
        k=num_parents
    )
    return parents

# --- GA Function: crossover ---
def crossover(parent1, parent2):
    """
    Create a child route using Ordered
Crossover (OX1).

```

A random slice is taken from parent1, and the rest is filled from parent2.

```
"""
child = [None] * len(parent1)

# Choose a random slice from parent1
start, end =
sorted(random.sample(range(len(parent1)), 2))

# Copy the slice from parent1 to the child
child[start:end] = parent1[start:end]

# Fill the remaining cities from parent2
parent2_pointer = 0
for i in range(len(child)):
    if child[i] is None:
        while parent2[parent2_pointer] in
child:
            parent2_pointer += 1
        child[i] = parent2[parent2_pointer]
        parent2_pointer += 1

return child

# --- GA Function: crossoverPopulation ---
def crossover_population(parents,
offspring_size):
    """Create 'offspringSize' children using
crossover."""
    offspring = []
    # Ensure there are always enough parents
for the next crossover operation
    num_parents = len(parents)
    for i in range(offspring_size):
        # Pick two distinct parents for crossover
        parent1 = parents[i % num_parents]
        parent2 = parents[(i + 1) % num_parents]
        offspring.append(crossover(parent1,
parent2))
    return offspring
```

```

# --- GA Function: mutatePopulation ---
def mutate_population(population,
                      mutation_rate):
    """
    For each route, with a given probability,
    swap two random cities.
    This is known as Swap Mutation.
    """
    mutated_pop = []
    for route in population:
        mutated_route = route[:] # Create a copy
        if random.random() < mutation_rate:
            # Select two random indices to swap
            idx1, idx2 =
random.sample(range(len(mutated_route)), 2)
            # Swap the cities
            mutated_route[idx1],
mutated_route[idx2] = mutated_route[idx2],
mutated_route[idx1]
            mutated_pop.append(mutated_route)
    return mutated_pop

# --- Main GA Function: geneticAlgorithm ---
def genetic_algorithm(pop_size,
                     max_generations, num_parents, mutation_rate,
                     patience):
    """The main function to run the genetic
    algorithm for TSP."""

    # Initialize population and evaluate its
    fitness
    population = initialize_population(pop_size,
NUM_CITIES)
    fitness_scores =
calculate_all_fitness(population)

    # Track the best route found so far
    best_fitness_idx =
fitness_scores.index(max(fitness_scores))
    best_route = population[best_fitness_idx]
    best_distance = tour_distance(best_route)

```

```

unchanged_generations = 0

print(f"Initial best distance:
{best_distance:.2f}")

for gen in range(max_generations):
    # Select parents
    parents = select_mating_pool(population,
fitness_scores, num_parents)

    # Create offspring via crossover
    offspring_size = pop_size - len(parents)
    offspring = crossover_population(parents,
offspring_size)

    # Mutate the offspring
    mutated_offspring =
mutate_population(offspring, mutation_rate)

    # The new population is a combination of
the best parents (elitism) and the new offspring
    population = parents + mutated_offspring

    # Evaluate fitness of the new population
    fitness_scores =
calculate_all_fitness(population)

    # Find the best route in the current
generation
    current_best_idx =
fitness_scores.index(max(fitness_scores))
    current_best_route =
population[current_best_idx]
    current_best_distance =
tour_distance(current_best_route)

    # Update the overall best route if the
current one is better
    if current_best_distance < best_distance:
        best_distance = current_best_distance

```

```

        best_route = current_best_route
        unchanged_generations = 0
        print(f"Generation {gen + 1}: New
best distance = {best_distance:.2f}")
    else:
        unchanged_generations += 1

    # Stop if the solution hasn't improved for
many generations
    if unchanged_generations >= patience:
        print(f"\nStopping early after
{patience} generations without
improvement.")
        break

    return best_route, best_distance

if __name__ == "__main__":
    # GA Hyperparameters
    POP_SIZE = 100
    MAX_GENERATIONS = 500
    NUM_PARENTS = 20
    MUTATION_RATE = 0.02
    PATIENCE = 50 # For early stopping

    # Run the algorithm
    final_route, final_distance =
genetic_algorithm(
    pop_size=POP_SIZE,

max_generations=MAX_GENERATIONS,
    num_parents=NUM_PARENTS,
    mutation_rate=MUTATION_RATE,
    patience=PATIENCE
)
    print("\n" + "="*40)
    print("Genetic Algorithm Finished!")
    print(f"Optimal route found: {final_route}")
    print(f"Total distance: {final_distance:.2f}")
    print("="*40)

```